UNIVERSIDADE FEDERAL DO ABC

Carlos Eduardo Finatti Caldas - RA: 11202320384
Gustavo Bruschi Antunes - RA: 11202021998

Projeto Prático
Análise Comparativa de Algoritmos de Classificação

Inteligência Artificial

Santo André – SP

2026

Sumário

1 Introdução **3**
2 Referencial Teórico **4**
2.1 Aprendizado **4**
2.2 Modelos **4**
2.3 Aprendizado Hebbiano e Backpropagation **6**
2.4 Métricas **6**
3 Metodologia **8**
4 Resultados **10**
4.1 Base de Dados Wine **10**
4.2 Base de dados PSDAS **11**
4.3 Gráficos e Matrizes de confusão **12**
4.3.1 Matrizes de Confusão 12
4.3.2 Gráficos 13
5 Conclusões **15**
Referências Bibliográficas **16**

1 Introdução

Um tópico fundamental no estudo de aprendizado de máquina é o de algoritmos de classificação, pois permitem separar dados em classes, a partir de uma base prévia de dados, de modo a realizar previsões e análises a partir de padrões de maneira mais eficiente.

Diante disso, o objetivo desse trabalho é avaliar, de maneira prática, a performance de dois algoritmos de classificação em um estudo supervisionado, utilizando duas bases de dados previamente selecionadas: a base de dados Wine[1] e a base de dados Predict Students' Dropout and Academic Success[2].

Para essa avaliação, foram utilizados a arquitetura de Multi-Layer Perceptron (MLP) e o algoritmo de Random Forest. Seus respectivos desempenhos foram quantificados através das métricas de Revocação *(Recall)*, Acurácia, Precisão e F1-score, além da montagem das Matrizes de Confusão de ambos os algoritmos.

2 Referencial Teórico

2.1 Aprendizado

Tendo em vista que o objetivo desse trabalho é avaliar o desempenho dos algoritmos de classificação quanto a sua precisão, acurácia, F1-Score e revocação, torna-se evidente que é necessário uma base de comparação para quantificar essas métricas. Portanto, é preciso utilizar a técnica de aprendizado de máquina supervisionado[3] para criar essa base de comparação.

2.2 Modelos

Uma vez que essa base de comparação esteja estabelecida, diferentes modelos podem ser escolhidos para realizar a classificação dos dados, como Naive-Bayes, K-Nearest-Neighbors, Decision Tree, entre outros [3]. Por outro lado, existem as arquiteturas de redes neurais artificiais, nos quais se utilizam de dois ou mais neurônios artificiais. Em ambos os casos, os modelos escolhidos devem ser treinados através de uma base de dados fornecida. A base de dados deve ser segmentada em dados de treinamento e dados de teste, selecionados de maneira aleatória (ou pseudoaleatória) para evitar o enviesamento dos resultados.

Neste trabalho, foi escolhido o modelo clássico Random Forest. O algoritmo de Random Forest utiliza da união de um determinado número de algoritmos de Decision Tree, balanceando as saídas dessas árvores de decisão menores para compor uma saída única para a floresta [4].

Além disso, também, foi escolhido a arquitetura de Multi-Layer Perceptron, uma arquitetura de rede neural artificial de múltiplas camadas. Essa arquitetura é estruturada por um determinado número de neurônios artificiais interconectados, agrupados em uma cadeia de Single-Layer Perceptrons (SLP) em camadas.

Os neurônios artificiais são estruturas que recebem um vetor de entradas, que, posteriormente, é multiplicado vetorialmente com um vetor de pesos. O resultado desse produto vetorial é, em seguida, somado a um valor de viés (*bias*). Então, o vetor somatório é fornecido a uma função de

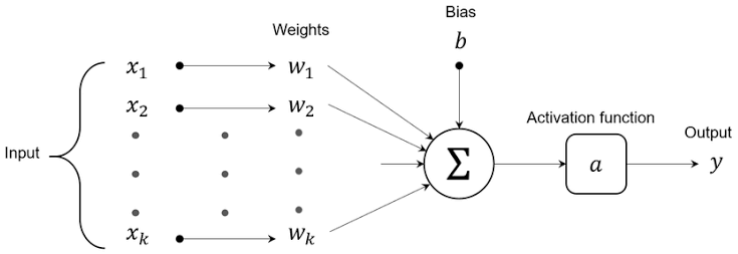
ativação que gera uma saída final. A Figura 1 representa um neurônio artificial. Segue a equação da saída do neurônio:

$$y = a(\vec{X}_{[1\times k]} \times \vec{W}_{[k\times 1]} + b) \tag{2.1}$$

O Single-Layer Perceptron é uma arquitetura de rede neural que utiliza de dois ou mais neurônios artificiais para compor um vetor de saída. No caso do MLP, o vetor de saída da primeira camada se torna o vetor de entrada da segunda camada em uma estrutura de cadeia de SLPs, como representado na Figura 2. A equação de saída segue uma forma similar a equação 2.1, com a alteração das dimensões dos vetores:

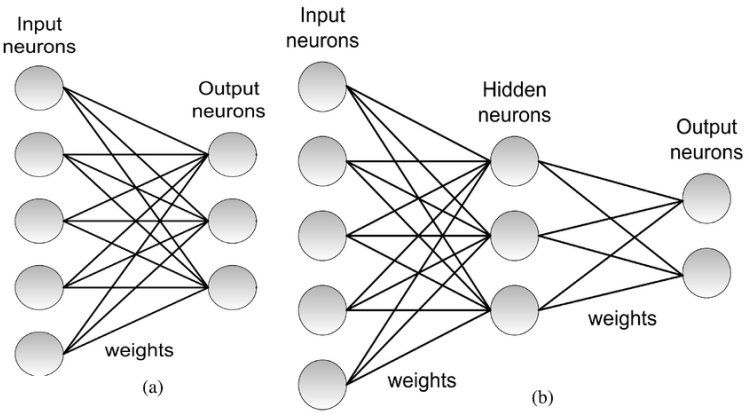
$$\vec{Y}_{[1\times N]} = a(\vec{X}_{[N\times k]} \times \vec{W}_{[k\times N]} + \vec{B}_{[1\times N]}) \tag{2.2}$$

Figura 1: Neurônio Artificial



Fonte: Ataei, M.; 2020 [5]

Figura 2: (a) SLP, (b) MLP



Fonte: Comuñas-Mesa, L.; Linares-Barranco, B.; Serrano-Gotarredona, T.; 2019 [6]

2.3 Aprendizado Hebbiano e Backpropagation

Para garantir a proporcionalidade dos pesos em relação à dependência da ativação de cada neurônio, utiliza-se a fórmula proposta por Hebb [7], que modifica os valores dos pesos em função dos erros de predição pelo modelo. O valor η representa a taxa de aprendizagem do modelo.

$$\Delta w_{i,j} = \eta(y_j - \hat{y_j})x_i \tag{2.3}$$

No caso dos perceptrons, a técnica de Backpropagation permite o treinamento da rede com base na variação do gradiente de erro em função da variação dos pesos, seguindo a equação:

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \Sigma} \cdot \frac{\partial \Sigma}{\partial w_i} \tag{2.4}$$

$$E = (\hat{y} - y)^2 \tag{2.5}$$

2.4 Métricas

A partir das previsões feitas pelo modelo, é possível montar a Matriz de Confusão para aquele algoritmo. Convencionalmente, as linhas da matriz de confusão representam as classes reais dos dados, enquanto as colunas representam as classes previstas pelo modelo [8]. Portanto, numa matriz $M_{3\times3}$, o elemento $a_{1,1}$ representa um dado que realmente pertencia à classe 1 e que o modelo foi capaz de categorizar corretamente, por exemplo.

Logo, é possível generalizar isso para uma matriz de confusão multiclasse $M_{m\times n}$, cujo elemento $a_{i,j}$ representa um verdadeiro positivo (TP) se $i = j$. Similarmente, é possível definir verdadeiros negativos (TN), falso positivos (FP) e falso negativos (FN) a partir da matriz de confusão.

Os falso positivos ocorrem quando o modelo prevê uma certa classe, enquanto, na realidade, o dado pertencia a outra. Ou seja, tomando $a_{i,j}$ como verdadeiro positivo, $i \in [1, m]$ e $j \in [1, n]$, os falso positivos são representados pelos elementos $a_{k,j} \mid k \in [1, m] \setminus \{i\}$. A mesma lógica se aplica para os falso negativos, apenas invertendo as colunas pelas linhas, ou seja, $a_{i,k} \mid k \in [1, n] \setminus \{j\}$.

Da matriz de confusão é possível extrair diferentes métricas, entre elas: acurácia, precisão, revocação (*recall*) e F1-Score, que são definidos pelas seguintes expressões:

$$accuracy = \frac{TP + TN}{TP + FP + FN + TN} \tag{2.6}$$

$$precision = \frac{TP}{TP + FP} = P \tag{2.7}$$

$$recall = \frac{TP}{TP + FN} = R \tag{2.8}$$

$$F1_{score} = \frac{2 \cdot P \cdot R}{P + R} \tag{2.9}$$

3 Metodologia

Primordialmente, é necessário escolher a arquitetura de rede neural a ser utilizada e o modelo de algoritmo. Neste trabalho, foram escolhidos o MLP e o algoritmo de Random Forest. Em seguida, é necessário escolher duas bases de dados de modo a tornar a coleta de dados mais diversa. Neste trabalho, foi utilizada a linguagem de programação Python através da ferramenta Google Colab.

A natureza das bases de dados é fundamental para o estudo, já que o objetivo é avaliar o desempenho do algoritmo e não classificar os dados em uma base de dados. Portanto, deve-se selecionar uma base de dados que já possui as classes determinadas (e.g. as espécies de flores da base Iris [9]). Para este projeto foram selecionadas as bases de dados Wine [1] e a base de dados Predict Student's Dropout and Academic Success (PSDAS), ambas com 3 classes distintas (1, 2, 3). No caso da base Wine, as classes representam três diferentes áreas de produção de vinho. Já no caso da base PSDAS, a classe 1 representa alunos evadidos, a classe 2 representa alunos matriculados e a classe 3 representa alunos graduados. [2].

Com as bases selecionadas, deve-se, então, realizar o tratamento dos dados. Inicialmente, realiza-se a divisão da base de dados em duas seções: a seção de treinamento e a seção de teste; nas quais os dados são atribuídos de maneira pseudoaleatória através de um gerador de números aleatórios e uma semente de valor 42. Na sequência, é feita a padronização dos dados através do objeto StandardScaler da biblioteca scikit-learn (sklearn), o cálculo da média e desvio padrão dos dados de treinamento e, por fim, a transformação dos dados aplicando o método ***transform()***, que subtrai a média e o desvio padrão dos dados de treinamento, de ambas as seções de dados.

```
scaler = StandardScaler()
xr_train = scaler.fit_transform(xr_train)
xr_test = scaler.transform(xr_test)
```

Listing 3.1: Tratamento de Dados

Após os dados serem tratados, cria-se os classificadores MLPClassifier e RandomForestClassifier das bibliotecas sklearn.neural_network e sklearn.ensemble respectivamente. Então, os dados

de treinamento são fornecidos aos modelos e o treinamento é realizado pelo método ***fit()***.

Cada algoritmo e arquitetura tiveram 3 variações de estrutura de modo a ser possível avaliar como a variação de hiperparâmetros pode alterar o desempenho nos testes. As variações do MLP foram de: 100 neurônios em uma única camada oculta (teste MLP-I), 30 neurônios em uma única camada oculta (teste MLP-II) e 100 neurônios na primeira camada oculta e 50 neurônios na segunda camada oculta (teste MLP-III). Já as variações do Random Forest foram de: 100 (RF-I), 50 (RF-II) e 200 árvores (RF-III) de decisão internas.

Uma vez que os modelos são treinados, utilizou-se o método ***predict()*** para realizar a avaliação dos dados de teste. Para o MLP foi estabelecido um limite de 500 iterações para a base Wine e 800 iterações para base PSDAS.

Após isso, é possível extrair as métricas fornecidas pelos métodos ***accuracy_score()*** e ***classification_report()***, com a matriz de confusão de cada modelo sendo gerada pelo método ***confusion_matrix()***.

```
mlp_wine = MLPClassifier(hidden_layer_sizes=(100,), max_iter=500, random_state
    =42)
mlp_wine.fit(x_train, y_train.values.ravel())

y_pred = mlp_wine.predict(x_test)
```

Listing 3.2: Treinamento e predição do MLP com 100 neurônios nas camadas ocultas

```
modelo = RandomForestClassifier(n_estimators=100, random_state=42)
modelo.fit(xr_train, yr_train.values.ravel())

yr_pred = modelo.predict(xr_test)
```

Listing 3.3: Treinamento e predição do Random Forest com 100 árvores.

4 Resultados

4.1 Base de Dados Wine

Para a base Wine, foram obtidos as seguintes acurácias baseadas nos testes descritos anteriormente:

Tabela 1: Acurácia dos modelos na base Wine

Modelo	Acurácia	Modelo	Acurácia
MLP-I-Wine	0,9722	RF-I-Wine	1,0000
MLP-II-Wine	0,9444	RF-II-Wine	1,0000
MLP-III-Wine	1,0000	RF-III-Wine	1,0000

O modelo MLP obteve, em cada um de seus testes os seguintes valores para Precisão, Recall e F1-Score de acordo com cada classe:

Tabela 2: Métricas dos MLP na base Wine

Teste MLP-I-Wine				Teste MLP-II-Wine				Teste MLP-III-Wine			
Classe	Precisão	Recall	F1-Score	Classe	Precisão	Recall	F1-Score	Classe	Precisão	Recall	F1-Score
1	1,00	1,00	1,00	1	1,00	1,00	1,00	1	1,00	1,00	1,00
2	0,93	1,00	0,97	2	0,88	1,00	0,93	2	1,00	1,00	1,00
3	1,00	0,90	0,95	3	1,00	0,80	0,89	3	1,00	1,00	1,00

Já o algoritmo de Random Forest obteve as seguintes métricas de Precisão, Recall e F1-Score, para cada classe:

Tabela 3: Acurácia do Random Forest na base Wine

Teste RF-I-Wine				Teste RF-II-Wine				Teste RF-III-Wine			
Classe	Precisão	Recall	F1-Score	Classe	Precisão	Recall	F1-Score	Classe	Precisão	Recall	F1-Score
1	1,00	1,00	1,00	1	1,00	1,00	1,00	1	1,00	1,00	1,00
2	1,00	1,00	1,00	2	1,00	1,00	1,00	2	1,00	1,00	1,00
3	1,00	1,00	1,00	3	1,00	1,00	1,00	3	1,00	1,00	1,00

4.2 Base de dados PSDAS

Similarmente, para a base de dados PSDAS foram obtidos os seguintes dados:

Tabela 4: Acurácia dos modelos na base PSDAS

Modelo	Acurácia	Modelo	Acurácia
MLP-I-PSDAS	0,6994	RF-I-PSDAS	0,7638
MLP-II-PSDAS	0,7118	RF-II-PSDAS	0,7627
MLP-III-PSDAS	0,7096	RF-III-PSDAS	0,7672

O modelo MLP obteve as seguintes métricas em seus testes:

Tabela 5: Métricas do MLP na base PSDAS

Teste MLP-I-PSDAS				Teste MLP-II-PSDAS				Teste MLP-III-PSDAS			
Classe	Precisão	Recall	F1-Score	Classe	Precisão	Recall	F1-Score	Classe	Precisão	Recall	F1-Score
Dropout	0,74	0,70	0,72	Dropout	0,74	0,70	0,72	Dropout	0,72	0,73	0,72
Enrolled	0,36	0,32	0,34	Enrolled	0,40	0,38	0,39	Enrolled	0,41	0,45	0,43
Graduate	0,78	0,84	0,81	Graduate	0,80	0,84	0,82	Graduate	0,83	0,79	0,81

O algoritmo de Random Forest obteve:

Tabela 6: Métricas do Random Forest na base PSDAS

Teste RF-I-PSDAS				Teste RF-II-PSDAS				Teste RF-III-PSDAS			
Classe	Precisão	Recall	F1-Score	Classe	Precisão	Recall	F1-Score	Classe	Precisão	Recall	F1-Score
Dropout	0,80	0,75	0,77	Dropout	0,80	0,76	0,78	Dropout	0,80	0,76	0,78
Enrolled	0,56	0,35	0,43	Enrolled	0,54	0,35	0,42	Enrolled	0,56	0,34	0,42
Graduate	0,78	0,92	0,75	Graduate	0,79	0,92	0,85	Graduate	0,79	0,93	0,85

Os resultados obtidos mostram que, de modo geral, o algoritmo de Random Forest teve um desempenho melhor que o MLP, já que seus valores de acurácia, precisão, recall e F1-Score foram maiores na maioria dos casos. No entanto, é possível ver que, no caso da base PSDAS, o algoritmo de Random Forest obteve uma discrepância maior entre seus dados de precisão e recall, ao contrário do MLP, que os mantiveram mais equilibrados.

Além disso, é possível observar que a variação de hiperparâmetros nos modelos afeta diretamente e expressivamente os resultados. Em particular, é possível observar que no caso do MLP na base PSDAS, foi obtido um melhor resultado de acurácia no teste II com 30 neurônios por camada que nos demais, enquanto o algoritmo Random Forest manteve um caráter crescente de acurácia com o aumento da quantidade de árvores (Tabela 4).

4.3 Gráficos e Matrizes de confusão

Esta seção contém os gráficos obtidos, através da biblioteca matplotlib.pyplot, a partir dos dados da seção anterior e as matrizes de confusão de cada teste e modelo.

4.3.1 Matrizes de Confusão

Tabela 7: Matrizes de confusão do MLP na base Wine

MLP-I-Wine

Real\Predito	1	2	3
1	14	0	0
2	0	12	0
3	0	1	9

MLP-II-Wine

Real\Predito	1	2	3
1	14	0	0
2	0	12	0
3	0	2	8

MLP-III-Wine

Real\Predito	1	2	3
1	14	0	0
2	0	12	0
3	0	0	10

Tabela 8: Matrizes de confusão do Random Forest na base Wine

RF-I-Wine

Real\Predito	1	2	3
1	14	0	0
2	0	12	0
3	0	0	10

RF-II-Wine

Real\Predito	1	2	3
1	14	0	0
2	0	12	0
3	0	0	10

RF-III-Wine

Real\Predito	1	2	3
1	14	0	0
2	0	12	0
3	0	0	10

Tabela 9: Matrizes de confusão do MLP na base PSDAS

MLP-I-PSDAS

Real\Predito	1	2	3
1	198	46	40
2	44	51	64
3	26	46	370

MLP-II-PSDAS

Real\Predito	1	2	3
1	200	47	37
2	43	60	56
3	28	44	370

MLP-III-PSDAS

Real\Predito	1	2	3
1	207	49	28
2	44	71	44
3	37	55	350

Tabela 10: Matrizes de confusão do Random Forest na base PSDAS

RF-I-PSDAS

Real\Predito	1	2	3
1	213	22	49
2	40	56	63
3	13	22	407

RF-II-PSDAS

Real\Predito	1	2	3
1	215	21	48
2	43	55	61
3	11	26	405

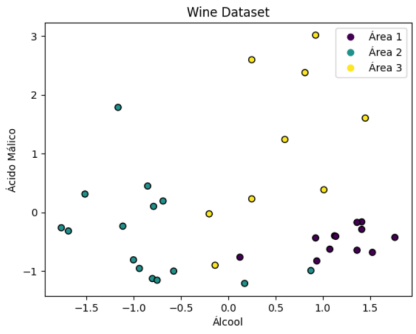
RF-III-PSDAS

Real\Predito	1	2	3
1	215	23	46
2	42	54	63
3	12	20	410

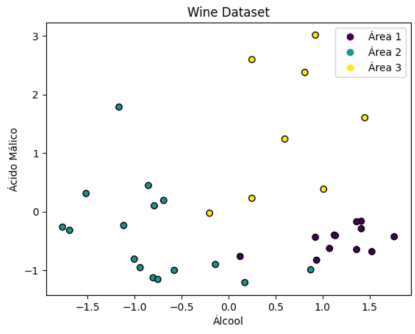
4.3.2 Gráficos

Figura 3: Resultados do MLP na base Wine

MLP-I-Wine



MLP-II-Wine



MLP-III-Wine

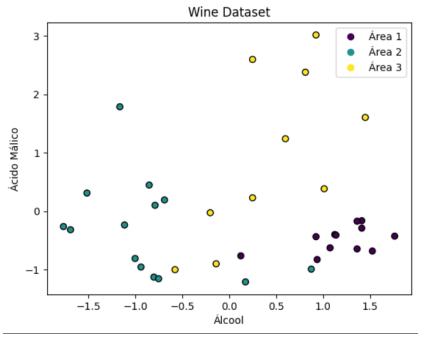
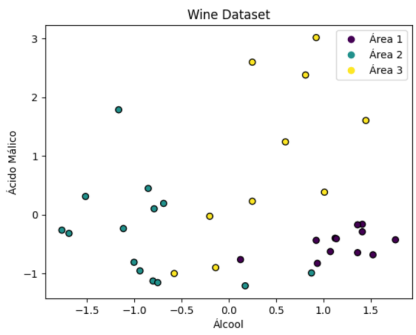
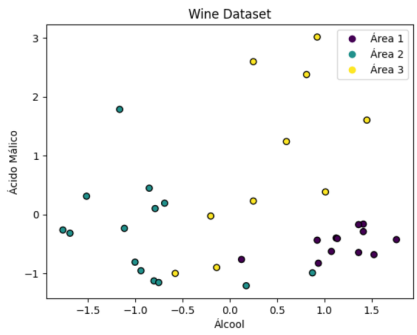


Figura 4: Resultados do Random Forest na base Wine

RF-I-Wine



RF-II-Wine



RF-III-Wine

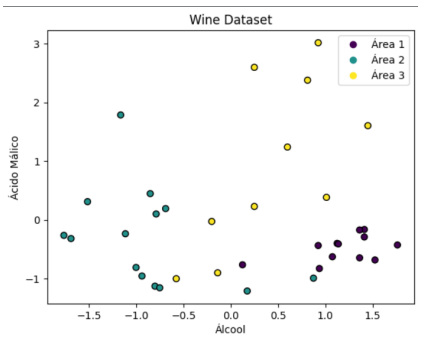
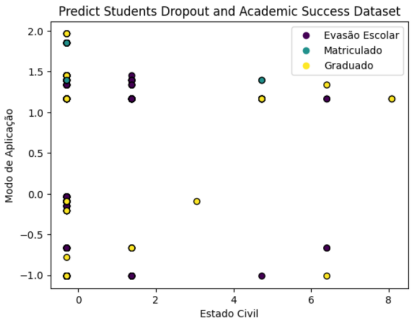
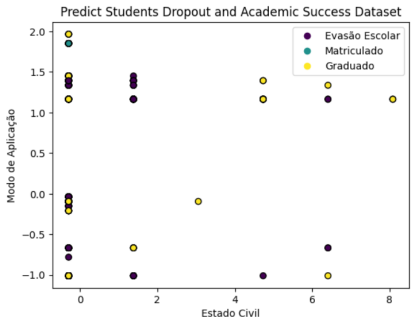


Figura 5: Resultados do MLP na base PSDAS

MLP-I-PSDAS



MLP-II-PSDAS



MLP-III-PSDAS

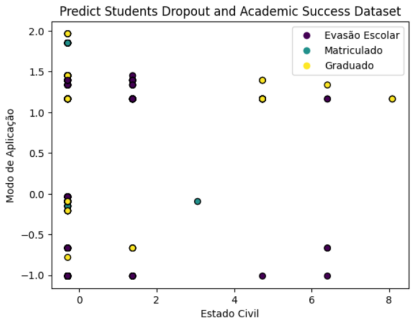
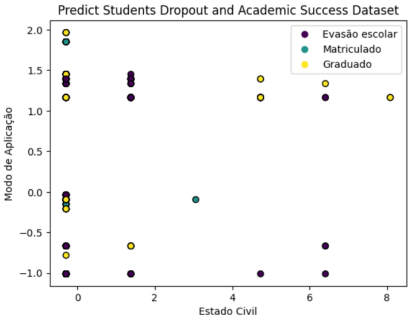
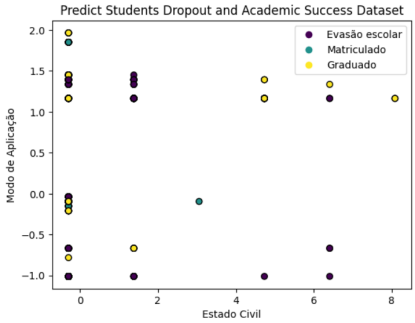


Figura 6: Resultados do Random Forest na base PSDAS

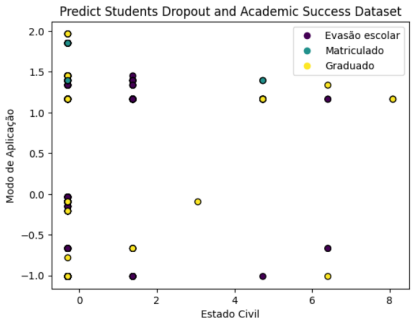
RF-I-PSDAS



RF-II-PSDAS



RF-III-PSDAS



5 Conclusões

Tendo em vista que o objetivo desse trabalho é avaliar o desempenho dos algoritmos e arquiteturas em questão (Random Forest e MLP, respectivamente), pode-se concluir que a escolha das bases de dados foi fundamental para que isso fosse possível. As bases Wine e PSDAS que possuem um caráter tabular, com três classes já definidas cada uma, permitiram que o estudo supervisionado fosse feito.

Ademais, a seleção e pré-processamento dos dados foi de suma importância para que os algoritmos pudessem ser executados de maneira conclusiva e efetiva, já que a utilização das bases, que estavam ordenadas inicialmente, geraria um viés de seleção nos modelos. Além disso, a variação dos hiperparâmetros de cada modelo nos respectivos testes (I, II e III) possibilitou uma análise mais profunda do desempenho dos algoritmos.

Da variação dos hiperparâmetros, é possível concluir que o desempenho do Random Forest aumenta conforme o número de árvores de decisão aumentam. Porém, fica evidente que o modelo pode causar erros como no caso dos falsos negativos, que aumentou do teste I para o III, mesmo que a acurácia seja maior. Também, é evidente que o aumento pelos resultados do MLP na base PSDAS que, mesmo com um número maior de neurônios e mais camadas, um resultado pior foi obtido em termos de acurácia, o que leva à conclusão de que a complexidade e tamanho da rede neural não implica que seu desempenho é superior. Uma possível causa disso é o fenômeno de *Overfitting*, no qual uma rede neural tem um desempenho reduzido devido ao acúmulo de ruído no seu treinamento, o que não ocorre numa rede neural menor, que tem uma menor sensibilidade a esses ruídos, permitindo uma generalização maior[10].

Portanto, é possível verificar que o algoritmo de Random Forest possui uma acurácia maior de modo geral que a rede neural MLP, quando avaliando bases de dados tabulares. Além disso, os hiperparâmetros possuem um grande papel na classificação dos dados, permitindo uma maior performance quando estabelecidos de maneira precisa, de modo a evitar fenômenos como o *Overfitting*.

Referências Bibliográficas

1 AEBERHARD, S.; FORINA, M. *Wine*. 1992. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5PC7J.

2 REALINHO VALENTIM, V. M. M. M. J.; BAPTISTA, L. *Predict Students' Dropout and Academic Success*. 2021. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5MC89.

3 Wikipedia contributors. *Supervised learning — Wikipedia, The Free Encyclopedia*. 2026. <https://en.wikipedia.org/w/index.php?title=Supervised_learning&direction=next&oldid=1378413836>. [Online; accessed 7-November-2026].

4 Wikipedia contributors. *Random forest — Wikipedia, The Free Encyclopedia*. 2026. <https://en.wikipedia.org/w/index.php?title=Random_forest&oldid=1373673788>. [Online; accessed 7-November-2026].

5 ATAEI, M. et al. *NPLIC: A Machine Learning Approach to Piecewise Linear Interface Construction*. 2020.

6 CAMUÑAS-MESA, L.; LINARES-BARRANCO, B.; SERRANO-GOTARREDONA, T. Neuromorphic spiking neural networks and their memristor-cmos hardware implementations. *Materials*, v. 12, p. 2745, 08 2019.

7 ROSENBLATT, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, v. 65, n. 6, p. 386–408, 1958.

8 Wikipedia contributors. *Confusion matrix — Wikipedia, The Free Encyclopedia*. 2026. <https://en.wikipedia.org/w/index.php?title=Confusion_matrix&oldid=1382483577>. [Online; accessed 28-September-2026].

9 FISHER, R. A. *Iris*. 1936. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C56C76.

10 scikit-learn developers. *Cross-validation: evaluating estimator performance*. 2026. Acesso em: 8 nov. 2026. Disponível em: <https://scikit-learn.org/stable/modules/cross_validation.html>.